INDEX

NAME: A. Sella Saaikrishnan STD.: _____ SEC.: _____ ROLL NO.: _____ SUB.: DAA observation

S. No.	Date	Title	Page No.	Teacher's Sign / Remarks
		I: COUNTER METHOD		
1.	6/8/26	Finding Complexity - 1		
2.	6/8/26	Finding Complexity - 2		
3.	6/8/26	Finding Complexity - 3		
4.	6/8/26	Finding Complexity - 4		
5.	6/8/26	Finding Complexity - 5		
		II: Divide and Conquer method		
6	27/8/26	Find the no of zeros in a array		
7	27/8/26	Majority element		
8	27/8/26	Finding floor value		
9	27/8/26	Two elements sum to x.		
10	27/8/26	Sorting (Quick sort)		
		III. Greedy Method		
11.	27/8/26	Coin Problem		
12.	27/8/26	Cookies Problem		
13.	27/8/26	Burger problem		
14.	27/8/26	Sum max problem		
15.	27/8/26	Product of array elements.		

Scanned with OKEN Scanner

S. No.	Date	Title	Page No.	Teacher's Sign / Remarks
		IV Dynamic Programming		
1.	2/8/26	Playing with Numbers		
2.	2/8/26	Playing with chessboard		
3.	2/8/26	longest common subsequ		
4.	2/8/26	Longest non-decreasing subsequence		
		V Competitive Programming		
1.	10/10/26	1-Finding duplicates o(n^2) Time complexity O(1)		
2.	10/10/26	2-Finding duplicates o(n) Time complexity O(1)		
3.	10/10/26	3-Print Intersection of 2 sorted arrays- o(m*n) Time Complexity		
4.	10/10/26	4-Intersection of 2 sorted arrays o(m+n) time complexity O(1) space complexity.		
5.	10/10/26	5-Pair with attribute O(n^2) time complexity O(1) space complexity.		

Completed

6/8/26
Ex-1

Finding Complexity Using Counter Method

Aim:

To find the counter variable and time complexity.

Algorithm:

```
void function (int n)
{
  int i = 0;
  int s = 1;
  while (s <= n)
  {
    i++;
    s += i;
  }
}
```

Program:

```
#include<stdio.h>
int main()
{
  int n, count = 0;
  Scanf ("%d", &n);
  int s = 1 ; count ++;
  int i = 1 ; count ++;
  while (s <= n, count ++)
  {
    i++ ; count++;
    s+=i ; count ++;
  }
  Printf ("%d", count);
}
```

Time complexity:

Linear loop, hence O(n)

Sample Input	Sample output	Got
9	12	12
4	9	9

o/p ✓✓

Result:

Thus the time complexity and counter variable is found for the given algorithm.

Scanned with OKEN Scanner

6/8/26

Ex:2 Finding Complexity using Counter Method - 2

Aim:

To find the time complexity for a given algorithm using Counter Method.

Algorithm:

#

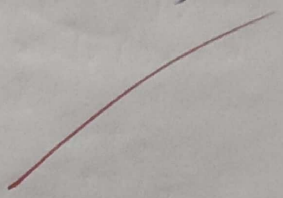
```
{
  if(n==1)
  {
    Printf("*");
  }
  else
  {
    for(int i=1; i<=n; i++)
    {
      for(int j=1; j<=n; j++)
      {
        Printf("*");
        Printf("*");
        break;
      }
    }
  }
}
```

Program:

```
#include<stdio.h>
int main()
{
  int n;
  Scanf("%d", &n);
  int count=0;
  if(n==1)
  {
    count++;
  }
  else
  {
    count++;
    for(int i=1; i<=n; i++)
    {
      count++;
      for(int i=1; i<=n; i++)
```

Scanned with OKEN Scanner

```
        count++;
        for(int j=1;j<=n;j++)
        {
            count++;
            count++;
            count++;
            break;
        }
        count++;
    }
}
```



Result:

Input	Expected	Got
2	12	12
1000	5002	5002
143	717	717

6/8/26

Ex:3 Finding Complexity using Counter Variable Method - 3

Aim:

To find the counter variable and complexity for the given algorithm.

Algorithm:

```
for (int i=1; i<=num; ++i)
{
  if (num%i==0)
  {
    Printf("%d", i);
  }
}
```

Program:

```
#include <stdio.h>
int main()
{
  int num;
  Scanf("%d", &num);
  int count =0;
  int i=0;
  for (i=1; count++, i<=num; ++i, Counter++)
  {
    if (num%i==0)
      {
        count++;
      }
    }
    Printf("%d", Counter);
  }
```

Sample input	Sample output	Got
12	31	31
25	54	54

Time Complexity

O(n): Single for loop

Result:

Thus the time complexity and counter variable is found for the given algorithm.

Scanned with OKEN Scanner

6/8/26

Ex-4 Finding Complexity using Counter Method - 4

Aim:

To find the time Complexity and counter variable for the given algorithm.

Algorithm:

```
Void func(int n)
{
  int c=0;
  for(int i=n/2; i<n; i++)
    for(int j=1; j<n; j=2*j)
      for(int k=1; k<n; k=k*2)
        c++
}
```

Program:

```
#include<stdio.h>
int main()
{
  int n;
  Scanf("%d", &n);
  function(n);
  Printf("%d", count);
  return 0;
}
```

```
int c=0
for(int i=n/2; i<n; i++)
  for(int j=1; j<n; j=2*j)
    for(int k=1; k<n; k=k*2)
      c++
```

Input	Expected	Got
4	30	30
10	212	212

Result:

Thus the time complexity and counter variable is found for the given algorithm.

Scanned with OKEN Scanner

Ex:5 Finding Complexity using Counter Method -5

Aim:

To find the input time complexity and Counter variable for the given algorithm.

Algorithm:

```
Void reverse (int n)
{
int rev = 0, remainder;
while (n != 0)
{
  remainder = n % 10;
  rev = rev * 10 + remainder;
  n /= 10;
}
Print(rev);
}
```

Program:

```
#include <stdio.h>
int count = 1;
Void reverse (int n) {
    int rev = 0, remainder;
    Count++;
        while (n != 0) {
        Count++;
        remainder = n % 10;
        Count++;
        rev = rev * 10 + remainder;
        Count++;
        n /= 10;
        Count++;
    }
    Count++;
}
```

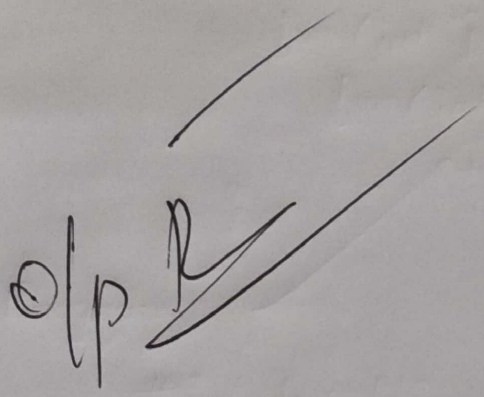
Scanned with OKEN Scanner

```
int main()
{
    Printf("%d", count);
}
```

Sample input	Sample output	Got
12	11	11

Result :

Thus the time complexity and Counter variable found for the given algorithm.



11. Double and Longest Method

Ex:1 No of zeros in a given array

Aim:

To find the no of zeros in a given array

Program:

```
#include <stdio.h>
int main()
{
int n;
scanf("%d", &n);
int a[n];
for(int i=0; i<n; i++)
{
scanf("%d", &a[i]);
}
int count = 0;
for(int i=0; i<n; i++)
{
if(a[i]==0)
{
count++;
}
}
Printf("%d", count);
return 0;
}
```

O/P

Input	Expected	Got
5	2	2
1		
1		
1		
0		
0		

Scanned with OKEN Scanner

10 0 0



Result:

Thus the prg to find no of zeroes in a given array is executed successfully.

Scanned with OKEN Scanner

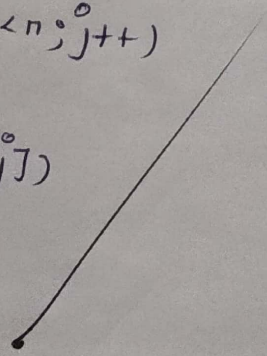
Ex 2. Majority Element

Aim:

To write a program to find the majority element.

Program:

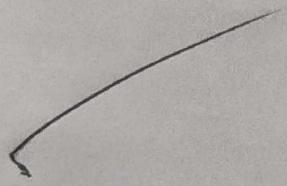
```
#include<stdio.h>
int main()
{
int n;
Scanf("%d", &n);
int a[n];
for(int i=0; i<n; i++)
{
Scanf("%d", &a[i]);
}
int ct=0;
for(int i=0; i<n; i++)
{
for(int j=0; j<n; j++)
{
if(a[i]==a[j])
{
ct++;
}
}
}
if(ct>n/2)
{
Printf("%d", a[i]);
break;
}
}
```



Scanned with OKEN Scanner

output

Input	Expected	Got
3 3 2 3	3	3



Result:

The program is executed successfully.

Scanned with OKEN Scanner

Ex 3: Finding floor value

Aim:

To find the floor value of the given array.

Program:

```
#include <stdio.h>
int main()
{
int n;
scanf("%d", &n);
int a[n];
for (int i=0; i<n; i++)
{
scanf("%d", &a[i]);
}
int key;
scanf("%d", &key);
for (int i=n-1; i>=0; i--)
{
if (a[i]<=key)
{
Printf("%d", a[i]);
break;
}
}
}
```

Scanned with OKEN Scanner

output

Input	Expected	Got
6 1 2 8 10 12 19 5	2	2

Result

Thus the program is executed successfully.

Scanned with OKEN Scanner

Ex 4: Two Elements sum to X

Aim:

To find two elements sum to X.

Program:

```
#include<stdio.h>
int main()
{
int n;
scanf("%d", &n);
int a[n];
for(int i=0; i<n; i++)
{
scanf("%d", &a[i]);
}
int key;
int c=0, d=0;
scanf("%d", &key);
for(int i=0; i<n; i++)
{
for(int j=0; j<n; j++)
{
if(a[i]+a[j] == key)
{
c = a[i];
d = a[j];
break;
}
}
}
int b[2];
if(c<d)
{
b[0] = c;
b[1] = d
}
```

Scanned with OKEN Scanner

```
if (c!=0 && d!=0)
{
Printf("%d\n%d", b[0], b[1]);
}
else
{
printf("NO");
}
}
```

output

Input	Expected	Got
4	4	4
2	10	10
4		
8		
10		
14		

Result:

Thus the program is executed successfully.

Ex 5: Implementation of Quick Sort

Aim:

To write a program to implement quick sort.

Program:

```
#include <stdio.h>
void qsort (int a[], int l, int h)
{
  if (l<h)
  {
   if i=l;
   if j=h;
  int pivot = a[l];
  while (i<j)
  {
   while(a[i]<=pivot && i<h)
   {
     i++;
   }
   while (a[j] >pivot)
   {
      j--;
   }
   if (i<j)
   {
    int temp =a[i]
     a[i] =a[j];
     a[j] = temp;
   }
  }

    a[i] = a[j];
    a[j] = pivot;
    qsort(
```

Scanned with OKEN Scanner

```
int main()
{
    int n, i;
    int a[100];
    scanf("%d", &n);
    for(int j=0; j<n; j++)
    q sort(a,0,n-1);
    for(i=0; i<n; i++)
    {
      Printf("%d", a[i]);
    }
    return 0;
}
```

Input	Expected	Got
5 67 34 12 98 78	12 34 67 78 98	12 34 67 78 98

Result.

Thus the program is executed successfully.

Scanned with OKEN Scanner

III. Greedy

Ex 1:
27/8/26

1. Coin Problem

Aim:

To write a program to find the minimum no. of coins or notes needed using greedy method.

Program:

```
#include<stdio.h>
int main(){
    int v;
    scanf("%d",&v);
    int denomination[]={1000,500,100,50,20,10,5,2,1};
    int count=0;
    for (int i=0;i<9;i++)
    {
        count+=v/denominations[i];
        v%= denominations[i];
    }
    Printf("%d", count);
    return 0;
}
```

Input	Expected	Got
49	5	5

Result:

Thus the program is executed successfully.

Scanned with OKEN Scanner

Ex 2
27/8/26

2. Cookies Problem

Aim:

To write a program to maximize the no. of your content children and output maximum number using greedy method.

Program:

```
#include<stdio.h>
int main()
{
  int n,m;
  Scanf("%d",&n);
  int g[n];
  for(int i=0;i<n;i++)
  {
    scanf("%d",&g[i]);
  }
  Scanf("%d",&m);
  int s[m], used[m];
  for(int i=0;i<m;i++)
  {
    Scanf("%d",&s[i]);
    used[i]=0;
  }
  int count=0;
  for(int i=0;i<n;i++)
  {
    for(int j=0;j<m;j++)
    {
      if(used[j]==0 && s[j]>=g[i])
      {
        count++;
        used[j]=1;
        break;
      }
    }
  }
  Printf("%d",count);
  return 0;
}
```

Input	Expected	Got
n: 2 1 2 3 1 2 3	2	2

Result:

Thus the program to find the maximum number using greedy method is executed successfully.

Scanned with OKEN Scanner

Ex-3
27/8/26

3. Burger Problem

Aim:

To write a program to solve burger problem using Greedy method.

Program:

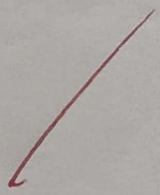
```
#include <stdio.h>
#include <math.h>
int main()
{
  int n;
  scanf("%d",&n);
  int arr[n];
  for(int i=0;i<n;i++)
  {
    scanf("%d",&arr[i]);
  }
  int temp;
  for(int i=0;i<n;i++)
  {
    for(int j=i+1;j<n;j++)
    {
      if(arr[i]<arr[j])
      {
        temp=arr[i];
        arr[i]=arr[j];
        arr[j]=temp;
      }
    }
  }
  int sum=0;
  for(int i=0;j<n;i++)
  {
    sum+=(pow(n,i)*arr[i]);
  }
  Printf("%d",sum);
  return0;
}
```

Scanned with OKEN Scanner

Test	Input	Expected	Got
3	18	18	18
1 3 2			

Result:

Thus the program was executed successfully.



Scanned with OKEN Scanner

4. Sum max problem

Aim

To write a Program to maximize the Sum of the given array.

Program:

```
#include<stdio.h>
#include<stdlib.h>
int cmp(const void *a, const void *b)
{
   return *(int *a)-*(int *)b;
}
int main()
{
   int n;
   scanf("%d",&n);
   int a[n];
   for(int i=0;i<n;i++)
   {
      scanf("%d",&a[i]);
   }
   qsort(a,n,sizeof(int),cmp);
   int sum=0;
   for(int i=0;i<n;i++)
   {
      sum+=a[i]*i;
   }
   Printf("%d",sum);
   return 0;
}
```

Input	Expected	Got
5 2 5 3 4 0	40	40

Result:

Thus the program is executed successfully using greedy method.

Scanned with OKEN Scanner

5. Product of Array elements - Minimum

m:

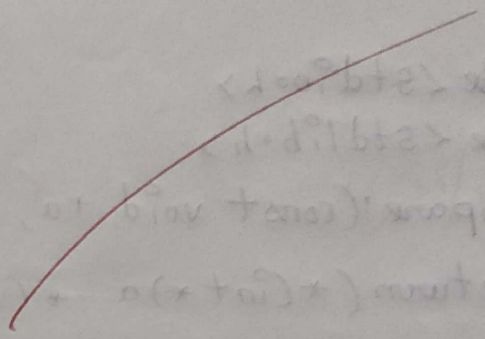
To write a program to arrange two arrays such that m of Pairwise Products is minimized.

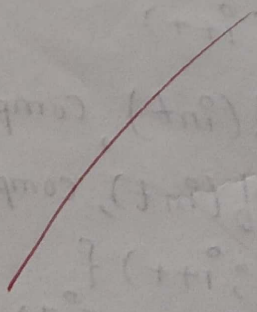
ogram:

```
#include<stdio.h>
#include<stdlib.h>
int Compare1(const void *a, const void *b){
    return(*(int*)a-*(int*)b);
}
int Compare2(cconst void *a, const void *b);
    return(*(int*)b-*(int*)a);
}
int main(){
    int n; sum=0;
    Scanf("%d",&n);
    int a[n], b[n];
    for(int i=0; i<n; i++)
    for(int i=0;} <n; i++)
    qsort(a,n,sizeof(int), Compare1)
    qsort(b,n, sizeof(int), compare2)
    for(int i=0; i<n; i++){
        sum+=(a[i]*b[i]);
    }
    Printf("%d", sum);
    return 0;
}
```

Sample Test case:

Input	Expected	Got
3 1 2 3 4 5 6	28	28





Result:

Thus the program is executed successfully.

Scanned with OKEN Scanner

Ex1
2/8/26

1. Playing with Numbers

Aim:

To write a c program to find the given task using dynamic programming.

Program.

```
#include <stdio.h>
int main() {
    int n;
    scanf("%d", &n);
    long long dp[n+1];
    dp[0] = 1;
    for(int i=1; i<=n; i++) {
        dp[i] = dp[i-1];
        if (i >= 3)
            dp[i] += dp[i-3];
    }
    Printf("%lld", dp[n]);
    return 0;
}
```

Result:

I/P

6

O/P

6

25 8641

Result

Thus the program is executed successfully.

Scanned with OKEN Scanner

Ex 2
2/8/26

2. Playing with chessboard

Aim:

To write a C program to find the given task using dynamic programming

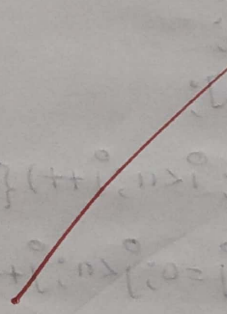
Program:

```
#include<stdio.h>
int main(){
    int n;
    Scanf("%d",&n);
    int a[n][n];
    int dp[n][n];
    for(int i=0;i<n;i++){
        for(int j=0;j<n;j++){
            Scanf("%d",&a[i][j]);
        }
    }
    dp[0][0]=a[0][0];
    for(int j=1;j<n;j++){
        dp[0][j]=dp[0][j]+a[0][j];
    }
    for(int i=1;i<n;i++){
        dp[i][0]=dp[i-1][0]+a[i][0];
    }
}
Printf("%d",dp[n-1][n-1]);
return 0;
}
```

Scanned with OKEN Scanner

Output

Input	Expected	Got
3 1 2 4 2 3 4 8 7 1	19	19



Result:

Thus the program is executed successfully.

Scanned with OKEN Scanner

3. Longest Common Subsequence

2/8/26

Aim:

To write a c program to execute the given task using dynamic programming.

Program:

```
#include <stdio.h>
#include <string.h>
int main() {
    char s1[100], s2[100];
    scanf("%d", s1);
    scanf("%s", s2);
    int n = strlen(s1);
    int m = strlen(s2);
    int dp[n+1][m+1];
    for (int i=0; i<=n; i++)
        dp[i][0]=0;
    for (int j=0; j<=m; j++)
        dp[0][j]=0;
    for (int i=1; i<=n; i++) {
        for (int j=1; j<=m; j++) {
            if (s1[i-1] == s2[j-1])
                dp[i][j] = dp[i-1][j-1]+1;
            else
                dp[i][j] = (dp[i-1][j] > dp[i][j-1])
        }
    }
```

Scanned with OKEN Scanner

```
    Printf("%d", dp[n][m]);
    return 0;
}
```

output

Input	Expected	Got
aab azb	2	2
ABCD ABCD	4	4

Result:

Thus the Program is executed successfully.

Scanned with OKEN Scanner

4: Longest non-decreasing subsequence

2/8/26

Aim:

To write a C program to execute a give task using dynamic programming.

Program:

```
#include <stdio.h>
int main() {
    int n;
    scanf("%d", &n);
    int a[n], dp[n];
    for (int i=0; i<n; i++) {
        scanf("%d", &a[i]);
        dp[i]=1;
    }
    int ans=1;
    for (int i=1; i<n; i++) {
        for (int j=0; j<n; j++) {
            if (a[j]<=a[i]) {
                if (dp[j]+1 > dp[i])
                    dp[i] = dp[j]+1;
                }
            }
            if (dp[i] > ans)
                ans = dp[i];
    }
    Printf("%d", ans);
    return 0;
}
```

Scanned with OKEN Scanner

output

Input	Excepted	Got
9 -1 3 4 5 22223	6	6
7 1 2 2 4 5 7 6	6	6

Result:

Thus the program is executed successfully

4. Space Complexity

1. Finding Duplicates - O(n^2) Time Complexity, O(1)

9/10/26

Aim:

To write a c program to execute the given task using space complexity.

Program:

```
#include<stdio.h>
int main(){
    int n;
    scanf("%d",&n);
    int arr[n];
    for(int i=0;i<n;i++){
        scanf("%d",&arr[i]);
    }
    for(int i=0;i<n;i++){
        for(int j=i+1;j<n;j++){
            if(arr[i]==arr[j]){
                Printf("%d",arr[i]);
            }
        }
    }
    return 0;
}
```

output

```
11
10 9 7 6 5 1 2 3 8 4 7
```

Result:

Thus the program is executed successfully.

Scanned with OKEN Scanner

2. Finding Duplicates

Aim:

To write a c program to execute the given task using time Complexity.

Program:

```
#include<stdio.h>
int main(){
    int n;
    Scanf("%d",&n);
    int arrn[n];
    for(int i=0;i<n;i++){
        Scanf("%d",&arrn[i]);
    }
    int visarrn[100]={0};
    for(int i=0;i<n;i++){
        if(visarrn[arrn[i]]==0)
            visarrn[arrn[i]]=1;
        else
            Printf("%d",arrn[i]);
    }
    return 0;
}
```

output

Input	Result
5 1 1 2 3 4	1

Result:

Thus the program is executed successfully.

Scanned with OKEN Scanner

3. Print intersection of 2 sorted arrays

10/10/26

Aim:

To write a program to execute the given task using using time complexity.

Program:

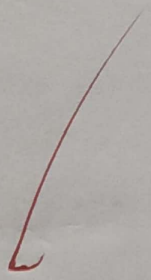
```
#include <stdio.h>
int main() {
    int T;
    Scanf ("%d", &T);
    While (T--) {
        int n1, n2;
        Scanf ("%d", &n1);
        int a[n1];
        for (int i=0; i<n1; i++)
            Scanf ("%d", &a[i]);
        Scanf ("%d", &n2);
        int b[n2];

    for (int i=0; i<n1; i++) {
        for (int j=0; j<n2; j++) {
            if (a[i] == b[j]) {
                Printf ("%d", a[i]);
                break;
            }
        }
    }

    Printf ("\n");
}
return 0;
}
```

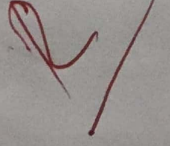
output

Input	Expected	Got
1 10 57 3 10 17 57 6 2 7 10 15 57 246	10 57	10 57



Result:

Thus the program is executed successfully.



Scanned with OKEN Scanner

4. Print Intersection of 2-Sorted arrays

0/10/26

Aim:

To write a C program to execute the given task using time complexity method.

Program:

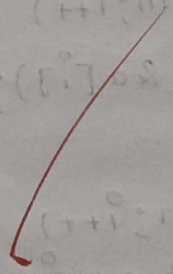
```
#include <stdio.h>
int main() {
    int T;
    Scanf("%d",&T);
    While(T--) {
        int m,n;
        Scanf("%d",&m);
        int a[m];
        For(int i=0;i<m;i++)
            Scanf("%d",&a[i]);
        int b[n];
        For(int i=0;i<n;i++)
            Scanf("%d",&b[i]);
        int i=0,j=0;
        while(i<m && j<n){
            if(a[i]==b[j]){
                Printf("%d ",a[i]);
                i++;
                j++;
            }
            else if(a[i]<b[j]){
                i++;
            }
```

Scanned with OKEN Scanner

```
        else {
            j++;
        }
    }
    Printf("\n");
}
return 0;
}
```

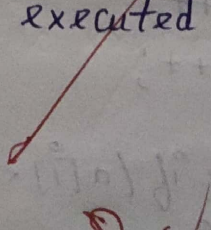
output

Input	Excepected	Got
1 3 10 17 57	10 57	10 57
6 2 7 10 15 57 246		



Result:

Thus the program is executed successfully.



Scanned with OKEN Scanner

5. Pair with Difference

Aim:

To write a C program to execute the given task using time complexity.

Program:

```
#include <stdio.h>
int main() {
    int n, k;
    Scanf("%d", &n);
    int A[n];
    for (int i=0; i<n; i++)
        Scanf("%d", &A[i]);
    Scanf("%d", &k);
    for (int i=0; i<n; i++) {
        for (int j=0; j<n; j++) {
            if (A[j] - A[i] == k) {
                Printf("1");
                return 0;
            }
        }
    }
    Printf("0");
    return 0;
}
```

output

Input	Expected	Got
3 1 3 5 4	1	1

Result:

Thus the program is executed successfully.

Scanned with OKEN Scanner

6. Pair with Difference

10/10/24

Aim:

To write a c program to execute the give program using time complexity method.

Program:

```
#include<stdio.h>
int main(){
    int n,k;
    Scanf("%d",&n);
    int A[n];
    for(int i=0;i<n;i++)
        Scanf("%d",&A[i]);
    Scanf("%d",&k);
    int i=0,j=1;
    while(i<n && j<n){
        if(i==j){
            j++;
        }
        else if(A[j]-A[i]==k){
            Printf("1");
            return 0;
        }
        else if(A[j]-A[i]<k){
            j++;
        }
        else{
            i++;
        }
    }
    Printf("0");
    return 0;
}
```

Scanned with OKEN Scanner

output

Input	Excepted	Got
3 1 3 5 4	1	1

Result:

Thus the program is executed successfully.

Scanned with OKEN Scanner